

DOCUMENTACION

Brenda Elizabeth Robles Concepcion

100067220

ISW 410, V20 SEMINARIO DE PROYECTOS I

Henry Marcelino Candelario Rosario

Estructura del proyecto

index.html = presentación del producto

archivo.html = Pantalla de carga del Excel

dashboard.html = Pantalla del dashboard (KPIs, gráficas, tabla)

CSS/

styles.css = global

JS/

archivo.js = Lógica de carga

dashboard.js = Lógica de lectura del Excel, render y acciones (PDF/correo)

Dependencias externas (CDN)

Librería	Uso
Tailwind CSS (cdn.tailwindcss.com)	Utilidades puntuales sobre el CSS propio
SheetJS xlsx@0.18.5	Leer el archivo Excel en el navegador
Chart.js	Gráficas de barras, dona y pastel
html2pdf.js	Exportar el dashboard a PDF
EmailJS (@emailjs/browser@3)	Enviar el resumen por correo sin backend

Flujo general de la aplicación

index.html ----- archivo.html ----- dashboard.html

(landing)

(subir Excel)

(visualizar)

index.html

Landing page estática con:

- **Navbar** con logo "GT" superpuesto (dos letras con posicionamiento absoluto) y nombre del producto.
- **Hero** con título y descripción del valor del producto.
- **Sección "¿Cómo funciona?"** con 3 tarjetas (Regístrate, Sube tu Excel, Dashboard).
- **CTA final** con botón que enlaza a archivo.html.

- **Footer** con crédito de autoría.

No contiene lógica JS propia.

archivo.html + archivo.js

Elementos clave del HTML

- `#zonaDrop`: `<label>` que envuelve el `<input type="file">`, actúa como zona de drag & drop.
- `#inputArchivo`: input oculto (`accept=".xlsx, .xls"`).
- `#iconoExcel`: ícono que aparece cuando ya hay un archivo cargado.
- `#textoPrincipal` / `#textoSecundario`: mensajes que cambian según el estado.
- `#nombreArchivo`: texto debajo de la caja con el nombre del archivo.
- `#btnAnalizar`: botón que dispara el procesamiento y la redirección.

Lógica en archivo.js

Función / bloque	Propósito
Listeners <code>dragenter/dragover/dragleave/drop</code>	Evitan el comportamiento por defecto del navegador (abrir el archivo) y dan feedback visual (borde verde al arrastrar, rosa al salir).
<code>inputArchivo.addEventListener("change", ...)</code>	Captura el archivo cuando se selecciona por clic.
<code>procesarArchivo(archivo)</code>	Valida el tipo MIME (<code>application/vnd.openxmlformats-officedocument.spreadsheetml.sheet</code> o <code>application/vnd.ms-excel</code>); si no es válido, alerta y limpia el input. Si es válido, guarda el archivo en <code>archivoTemporal</code> , actualiza textos, muestra el ícono y pinta el borde de verde.
Listener del botón <code>#btnAnalizar</code>	Si no hay <code>archivoTemporal</code> , alerta. Si hay archivo, usa <code>FileReader.readAsArrayBuffer</code> y en <code>onload</code> guarda el contenido como arreglo de bytes en <code>sessionStorage["archivoExcel"]</code> , luego navega a <code>dashboard.html</code> .

Detalle técnico importante: el archivo se serializa como `Array.from(new Uint8Array(contenido))` y se guarda con `JSON.stringify`. Esto es simple pero **poco eficiente para archivos grandes** (un Excel de unos pocos MB puede generar una cadena JSON varias veces más pesada, y `sessionStorage` suele tener un límite de 5 MB por origen).

dashboard.html + dashboard.js

Estructura del HTML

- **Navbar** (igual que en index.html).
- **.barra**: contiene #filtros (se llena dinámicamente con un <select> por columna) y los botones de acción (Descargar PDF, Enviar por correo, Generar otro).
- **#vacio**: mensaje que se muestra si no hay archivo cargado en sessionStorage.
- **#dashboardExportable**: contenedor que agrupa todo lo que se exporta a PDF:
 - #kpis: tarjetas de indicadores.
 - #graficas: contenedor de gráficas Chart.js.
 - #tabla: tabla con <thead>/<tbody> generados dinámicamente.
- **#modalCorreo**: modal para capturar el correo del destinatario y confirmar el envío.

Estado global (JS)

- let datosOriginales = []; // Todas las filas del Excel, sin filtrar
- let datosFiltrados = []; // Subconjunto según los filtros activos
- let columnas = []; // Nombres de las columnas (encabezados del Excel)
- let graficasActivas = []; // Instancias de Chart.js act

Paletas de color

- **PALETA_ESTADO**: asigna color de marca según palabras clave detectadas por *regex* en la etiqueta (abierto/pendiente → rosa, en proceso → ámbar, en espera/revisión → azul, cerrado/resuelto → verde, cancelado/rechazado → gris). Esto da consistencia visual sin importar cómo esté redactado el estado en el Excel.
- **PALETA_RESPALDO**: colores de respaldo (cíclicos) para etiquetas que no matchean ningún patrón conocido.
- **colorParaEtiqueta(etiqueta, indiceRespaldo)**: función que decide qué color usar para cada etiqueta.

Carga inicial — window.onload

1. Lee sessionStorage.getItem("archivoExcel").
2. Si no existe → mostrarVacio() (oculta el dashboard, muestra el mensaje de "no hay archivo").

B.R.

3. Si existe → reconstruye el Uint8Array, lo parsea con XLSX.read, toma la primera hoja (workbook.SheetNames[0]) y la convierte a JSON con sheet_to_json (usa defval: "" para rellenar celdas vacías).
4. Si hay datos: guarda en datosOriginales/datosFiltrados, deduce columnas a partir de las llaves del primer registro, configura defaults de Chart.js, genera los filtros y llama a actualizarTodo().
5. Cualquier error (Excel corrupto, formato inesperado) se captura y también muestra el estado vacío.

Procesamiento y agregación de datos

Función	Qué hace
contarValores(col)	Cuenta ocurrencias de cada valor único en una columna (celdas vacías se agrupan como "Sin dato").
esColumnaCategorica(col)	Heurística para decidir si una columna sirve para graficar: entre 2 y 12 valores únicos, y que no sean todos distintos (evita graficar, por ejemplo, un ID único por fila).
obtenerColumnasCategoricas()	Prioriza columnas cuyo nombre matchee estado, status, prioridad, tipo, categoría, área, departamento, empresa, cliente mediante regex; agrega el resto de columnas categóricas después; se queda con un máximo de 4 columnas para graficar.

Renderizado

Función	Qué hace
actualizarTodo()	Orquesta: contador, KPIs, gráficas y tabla.
renderContador()	Actualiza el texto "X de Y tickets".
renderKPIs()	Crea la tarjeta de "Total de tickets" y, si existe una columna de estado, hasta 3 tarjetas más con los valores más frecuentes de esa columna (con su color de marca y porcentaje).
crearTarjeta(...)	Genera el HTML de una tarjeta de KPI (número, ícono, color, subtítulo).
renderGraficas()	Destruye las gráficas anteriores (graficasActivas), obtiene hasta 4 columnas categóricas y crea una gráfica por columna: la primera es de barras, la segunda de dona, el resto de pastel. Usa contarValores para los datos y colorParaEtiqueta para los colores.

renderTabla()	Reconstruye encabezados (<thead>) según columnas y filas (<tbody>) según datosFiltrados. Si no hay resultados, muestra un mensaje.
---------------	--

Filtros

Función	Qué hace
crearFiltros()	Por cada columna, crea un <select> con todos los valores únicos (ordenados alfabéticamente) y un listener change que llama a aplicarFiltros.
aplicarFiltros()	Filtra datosOriginales exigiendo que todas las columnas con un <select> con valor seleccionado coincidan (AND lógico entre filtros). Actualiza datosFiltrados y llama a actualizarTodo().
limpiarFiltros()	Resetea todos los <select> a vacío y restaura datosFiltrados = datosOriginales.

Acciones globales (botones onclick)

Función	Qué hace
descargarPDF()	Usa html2pdf() sobre #dashboardExportable con orientación horizontal (landscape), tamaño A4 y escala 1.5 para el renderizado del canvas. El nombre del archivo incluye la fecha del día.
abrirModal() / cerrarModal()	Muestran/ocultan el modal de envío de correo.
generarOtro()	Borra sessionStorage["archivoExcel"] y regresa a archivo.html para cargar un nuevo Excel.

confirmarEnvio() — envío del reporte por correo

1. Valida el formato del correo ingresado con una regex simple (algo@algo.algo).
2. Deshabilita el botón mientras se procesa ("Procesando métricas...").
3. Construye tres bloques de texto dinámicos a partir de los datos filtrados:
 - o **Desglose de estados:** cuenta valores de la columna de estado y arma una frase tipo *"12 están abierto, 5 están cerrado"*.
 - o **Desglose por empresa/cliente:** busca una columna de empresa/cliente/área y calcula el porcentaje de cada una sobre el total.

- **Rango de fechas:** busca una columna de fecha y calcula el mínimo y máximo (ordenando como texto, no como fecha real — ver limitaciones).
- 4. Arma una tabla HTML con la distribución de estados.
- 5. Construye un correo HTML institucional completo (encabezado con gradiente, tarjeta de volumen total, tabla de distribución, párrafo de conclusión y pie de página).
- 6. Envía con `emailjs.send(EMAILJS_SERVICE_ID, EMAILJS_TEMPLATE_ID, templateParams)`, donde `templateParams` manda `to_email` y `html_body`.
- 7. En éxito: alerta de confirmación, cierra el modal y limpia el input. En error: alerta genérica y log en consola. En ambos casos (`finally`), reactiva el botón.

styles.css — resumen de convenciones

- **Variables CSS** (:root): --primary (rosa #D82299), --secondary (morado #3F2574), --blue (#4C9AFF), --radius y --transition.
- **Botones:** clases .btn-primary, .btn-main, .btn-correo, .btn-pdf, .btn-volver, .btn-subir, todas comparten padding/radius/transición base y luego cada una define su propio color de fondo.
- **Navbar** (.nav): fija arriba, con efecto *glassmorphism* (backdrop-filter: blur).
- **Hero** (.hero): fondo degradado oscuro con dos manchas de color difuminadas (::before/:after) simulando *blobs*.
- **Tarjetas del proceso** (.card, .circle): tarjetas con blur y números en círculo degradado.
- **Zona de carga** (.upload-box): borde punteado con efecto *shine* animado en :hover (::before con gradiente rotado).
- **Dashboard:**
 - .barra: barra sticky con filtros y botones.
 - .kpi / .tarjeta: grid responsivo de tarjetas de indicadores con barra superior de color y sombra.
 - .graficas / .grafica-box: grid de tarjetas contenedoras de cada gráfica (canvas dentro de .lienzo-wrap con altura fija de 260px).
 - .tabla-container / .tabla-scroll: tabla con encabezado *sticky* y scroll interno, bordes redondeados en el contenedor exterior.
 - .modal / .contenido-modal: modal centrado con overlay semitransparente.
- **Animación:** @keyframes fadeIn reutilizada en tarjetas y modal.

- **Responsive:** un único breakpoint en 768px que apila elementos en columna y ajusta paddings.

Resumen funcional (para no-técnicos)

1. El usuario entra a la landing y accede a la plataforma.
2. Sube un archivo Excel de tickets (arrastrando o seleccionando).
3. La app lee el archivo en el propio navegador (no se sube a ningún servidor) y genera automáticamente:
 - Tarjetas con el total de tickets y los estados más frecuentes.
 - Hasta 4 gráficas (barras, dona, pastel) según las columnas más relevantes del archivo (estado, prioridad, empresa, etc.).
 - Una tabla completa y filtrable con todos los tickets.
4. El usuario puede filtrar por cualquier columna, descargar todo como PDF, o enviar un resumen ejecutivo por correo a un Project Manager.
5. Para analizar otro archivo, el botón "Generar otro" limpia la sesión y regresa a la pantalla de carga.